

Go Runtime Internals Cheatsheet

L0 Essence: Go Runtime 的本质是通过基于 GMP 协程模型的轻量级用户态多路复用调度、基于 mcache/mcentral/mheap 三级架构的高效无锁内存分配、以及配合并发三色标记与混合写屏障自适应调优 (Pacer) 的垃圾回收引擎，构建自托管高并发语言运行时。

[M1.1] G、M、P 核心结构体物理定义与角色分工

• Goroutine (Goroutine): 用户态协程，包含专属物理栈空间 (起始 2KB)、程序计数器 PC、状态标记及调度上下文 (gobuf 结构体，保存 rsp/rip 寄存器)。
• Machine (Machine): 操作系统物理线程，负责绑定一个 P 执行 G 代码，内部保存有信号处理栈、当前活跃 G 的指针和所绑定的 P 的指针。
• Processor (Processor): 逻辑处理器/资源持有者，代表执行 Go 代码所需的 CPU 上下文，持有一个最大容量为 256 的本地可运行 G 队列 (runq) 及本地无锁内存分配缓存 mcache。

[M1.2] 调度主循环的核心决策链 schedule

当 M 执行调度方法 schedule() 时，其决策链如下：
1. 全局队列检查: 调度计数器每累加 61 次，就强行从全局 G 队列 (globalrunq) 中获取一个 G 执行，防止全局队列中的 G 发生饥饿。
2. 本地队列检查: 从当前 M 绑定的 P 本地运行队列 runq 中以无锁 CAS 方式弹出队列头部的 G 执行。
3. Netpoller 检查: 调用 netpoll 检查是否有网络 I/O 准备就绪的 G，并将其移入运行队列。
4. 工作窃取: 若仍未找到 G，调用 findrunnable() 开启工作窃取。

[M1.3] 任务工作窃取机制 Work Stealing

工作窃取是 Go 调度器实现负载均衡的核心：
• 当 P 本地可运行队列为空时，它会随机轮询其他 P，并尝试使用 CAS 锁从目标 P 的本地可运行队列中偷取一半 (最多 128 个) 的 Goroutines。
• 若所有 P 队列为空，则检查全局队列，然后检查 netpoll。
• 若全部落空，M 会释放 P 并进入“自旋 (spinning)”状态，最大自旋数量等于活跃 P 的个数，多余的 M 会进入休眠并挂载在 mheap.middle 链表上。

[M1.4] 网络轮询器 Netpoller epoll 汇流机制

网络轮询器实现了 Go 协程同步网络 API 与底层操作系统多路复用的对接：
1. 当 G 发起网络 I/O 阻塞调用 (如 conn.Read()) 时，Go 不会阻塞物理线程，而是将 socket 的文件描述符 fd 注册到系统多路复用器 (Linux 上为 epoll, macOS 上为 kqueue) 中。
2. 调用 gopark() 将当前 G 挂起，把 M 解绑并令其执行其他 G。
3. 在后台，调度循环或 sysmon 定期调用 netpoll 执行 epoll_wait。一旦 socket 就绪，对应的 G 会被标记为 _Grunnable 并送回 P 的运行队列。

[M1.5] Sysmon 监控线程与 SIGURG 信号抢占

sysmon 是一个不需要绑定 P、独占物理线程的运行时监控守护协程：
• 它以动态间隔周期运行 (10µs 到 10ms 不等)，负责检查长达 10ms 未被调度的 G，或者处于系统调用阻塞的 M。
• 在 Go 1.14 之前，Go 仅支持在函数入口插入 morestack 进行协作式抢占。
• Go 1.14+ 引入了基于信号的非协作式抢占：当 sysmon 检测到 G 运行超过 10ms，它会向运行该 G 的 M 发送 SIGURG 信号。M 的信号处理函数拦截当前执行流，保存现场寄存器，并强制注入 asyncPreempt 调用使 G 挂起并返回队列。

[M2.1] 内存分配三层物理架构 mcache/mcentral/mheap

Go 采用继承自 tcmalloc 的三级无锁细粒度锁分配架构以降低多线程竞争开销：
1. mcache (线程本地缓存): 每个 P 独占一个 mcache。由于 P 同时只能被一个 M 绑定执行，因此分配对象时完全无须加锁。
2. mcentral (全局中心缓存): 管理特定 Size Class 的内存跨度 (mspan)。分配时需要锁定特定的 mcentral 锁，避免全局大锁限制吞吐。
3. mheap (全局物理堆): 管理虚拟内存页；在底层将物理空间划分为 8KB 的 Page。当 mcentral 空间不足时，向 mheap 申请空闲 spans；mheap 不足时向 OS 发起系统调用 (如 mmap) 申请内存。

[M2.2] mspan 划分与 Size Classes 分配流程

Go 将堆内存按照 67 种不同大小的对象规格划分 (Size Classes，从 8 字节到 32KB 不等)：
• mspan 是内存管理的基本物理单位，由一个或多个连续的 8KB Page 组成，并被分割成特定 Class 大小的 Slot。
• 当调用 newobject 时，小对象 (<32KB) 根据大小匹配到 67 种 class 之一，并直接从本地 P 的 mcache.alloc[class] 中获取空闲 Slot。
• 若本地 span 已满，则从 mcentral 对应的 class 中置换一个有空闲 Slot 的 span；若 mcentral 亦空，则向 mheap 申请并切割新 Page。

[M2.3] 微小对象 Tiny Allocator 压缩分配

对于大小小于 16 字节且不包含任何指针的微小对象 (如小整型、小字符串结构)：
• Go 引入了 Tiny Allocator，它将多个微小对象的分配请求合并到 mcache 的单个 16 字节的物理内存块中。
• Tiny 分配器维护一个 16B 块的偏移量指针，新分配时仅需按对齐要求累加偏移，从而有效避免了小对象分配带来的严重元数据碎片和 Page 利用率低下问题。

[M2.4] 逃逸分析静态编译器逃逸场景判定

逃逸分析是 Go 编译器在静态编译 Passes 中执行的核心优化：
1. 编译器构建函数 AST 并推导变量指针的生命周期。
2. 逃逸场景: 若变量在函数返回后仍被外部引用 (作为返回值返回、赋值给全局变量、通过指针传入 channel、存储在 interface 变量中，或者变量尺寸过大 (如大数组)、或大小在编译期不确定)。
3. 逃逸的变量会被强行打上标记，运行时在堆上 (Heap) 进行分配；未逃逸的变量则直接在协程栈 (Stack) 上分配，随着函数退出瞬间无开销销毁。

[M2.5] 协程物理栈伸缩 copystack 连续栈拷贝

Go 抛弃了早期版本的分段栈 (Segmented Stack，存在严重的热裂点/Hot Split 问题)，采用 contiguous stack (连续栈) 架构：
• 每个 Goroutine 创建时仅分配 2KB 的栈空间。在每个函数入口处，编译器会插入几条汇编指令，比对当前 SP 寄存器与 g.stackguard0 判定栈是否溢出。
• 若溢出，触发 newstack()。Go 申请一块大小为原来 2 倍的连续物理内存，然后调用 copystack() 将旧栈的所有帧数据原封不动拷贝到新栈。
• 由于栈数据内含有指向本地变量的指针，Go 会遍历新栈并调整所有指针物理值使其偏移指向新栈对应物理地址，最后释放旧栈。

L1 Logical Framework

• 用户协程与队列分流: 协程 G 复用内核线程 M，逻辑处理器 P 充当资源中介，依靠本地双端队列和工作窃取 (Work Stealing) 算法避免全局锁竞争。
• 逃逸分析与栈堆分级: 编译器静态推导变量逃逸路径，未逃逸变量分配在随调用栈动态收缩的协程连续栈上，逃逸对象分配于无锁本地 Cache 细粒度堆上。
• 并发与写障保序: GC 在并发标记阶段应用三色标记法，通过 Dijkstra 插入屏障与 Yuasa 删除屏障构成的混合写屏障，在不停顿应用线程前提下保障并发安全性。
• 管道通道与接口去虚: 管道 Channel 利用原子锁环形缓冲区与 sudog 双向链表实现线程安全通信；动态接口通过 itab 反射，配合编译器去虚虚拟化分析，实现低开销间接调用。
• Data Flow Topology
• G Created → P Local Runqueue → M schedule() → G Run → Heap Alloc → Escape Analysis? Yes → mcache Alloc → Span Exhausted → mcentral Fetch → Channel Send → Buf Full? Yes → G blocked on sudog → M releases P → entersyscall → M detaches P → exitsyscall → G to global queue → Net I/O → Netpoller epoll_wait → wake up G → GC Start → STW Sweep Term → Write Barrier Active → Tricolor Mark → GC Assist Debt → Sweep white obj → Return memory to OS.

Trade-off Matrix: Scheduler & Memory

协程调度策略: 操作系 1:1 线程模型 (OS Threads) → 用户态 GMP 多路复用模型 (Go Runtimes) [1:1 模型由内核直接调度，能完美利用多核且支持全系统调用 → 但上下文切换涉及昂贵的特权级跳转 (微秒级)，且栈空间固定大 (8MB)，限制了高并发; GMP 在用户态完成协作式/非协作式抢占调度 (纳秒级)，栈起始仅 2KB，支持百万协程并发 [牺牲了单协程执行隔离性，换取了海量并发吞吐极速提升]]
垃圾回收策略: 分代拷贝垃圾回收 (Generational Copy GC) → 并发三色标记与写屏障机制 (Go Non-generational GC) [分代拷贝 GC 通过将新老对象物理分区并进行紧凑拷贝，吞吐极高且无内存碎片 → 但每次回收会导致较长的停顿时间 (STW); Go 三色标记利用混合写屏障在应用运行时并发标记，将停顿控制在微秒级，代价是无法物理整理碎片，且写屏障产生额外执行开销 [牺牲了垃圾回收的高吞吐，换取了极致的低延迟体验]]
内存分配架构: 传统全局锁内存堆分配 (Global Allocator) → 线程本地三级分块无锁分配 (tcmalloc 架构) [全局锁分配器实现简单，内存利用率相对紧凑 → 但在高并发多线程环境下，全局大锁会引发严重的 CPU 锁冲突; Go 的 mcache 绑定到逻辑处理器 P，分配小对象时无须任何锁，大大提升了多核并发性能，代价是增加了结构体复杂度与元数据内存开销 [牺牲了部分元数据内存空间，换取了高并发分配的无锁化极速响应]]
栈管理策略: 链式分段栈 (Segmented Stack) → 连续栈整体拷贝与指针调整 (Contiguous Stack) [分段栈在栈溢出时仅需分配小段链表挂载，分配极其迅速 → 但若在循环边界触发溢出与释放，会导致严重的“热裂点”性能崩溃; 连续栈在溢出时强行整体分配双倍连续内存并执行内存拷贝和指针重定向，虽然单词扩容开销大，但保证了栈内存的绝对物理连续性 & 寻址效率 [牺牲了单次栈扩容的开销，消除了热裂点并优化了常规执行效率]]

[M3.2] Dijkstra/Yuasa Go 混合写屏障

为了支持在并发标记阶段 (G 与 GC 线程同时修改指针) 保障强三色一致性并消除 STW 重新扫描栈的昂贵代价，Go 在 1.8 引入了 Hybrid Write Barrier (混合写屏障)：
1. Dijkstra 插入写屏障: 写入新指针时，强行将被指向的对象置灰，防止黑色对象指向白色对象被漏标。
2. Yuasa 删除写屏障: 删除旧指针时，强行将原先被引用的旧对象置灰，防止丢失可达路径。
3. 混合写屏障: 结合两者的优点，在写入时，将被覆盖的旧对象与新写入的对象均进行置灰，且所有在栈上新分配的对象在创建时直接标记为黑色。这使得栈上无须运行任何屏障指令，极大地保证了栈性能。

[M3.3] 用户辅助标记机制 GC Assist

在高并发大内存分配率场景下，若用户线程分配堆内存的速度快于 GC 线程标记存活对象的速度，堆内存会瞬间暴涨导致 OOM；\textbf{f1}.Go 引入了 GC Assist（辅助标记机制）。每个 G 都有一个内存分配债务账户。当 G 触发 mallocgc 分配内存时，系统根据当前的 GC 进度与分配速率，计算出该 G 欠下的 GC 标记工作量。当 GC 被强制暂停其用户逻辑，投入 CPU 时间片去协助 GC 扫描灰色对象，直到还清“债务”后才被允许返回分配并继续执行。

[M3.4] 并发清理阶段 Sweep 惰性扫描

在 Concurrent Sweep 阶段，Go 采用 Lazy Sweeping（惰性清理）来平滑 CPU 尖峰；\textbf{f1}.GC 不会在 STW 期间一次性扫描释放整个堆的所有垃圾页，这会造成可感知的停顿。当 Go 仅仅是将所有 span 的状态修改为“未清理（unswept）”，然后让用户线程继续运行。当应用协程后续调用 mallocgc 申请新内存而导致本地 local cache 耗尽时，该协程会主动去触发清理一个 unswept span 释放空间供自己使用，从而将清理开销平摊到每次分配中。

[M3.5] GC Pacer 自反馈水位线动态调整

GC Pacer 是 Go 垃圾回收的调优核心，旨在控制 GC 的触发时机；\textbf{f1}.它设置了一个触发水位线（Trigger Ratio，即堆大小增长到上一轮存活对象的多少比例时触发下一次 GC，默认 $G O G C = 100$ ）。当 Pacer 实时监控三个核心指标：上轮 GC 的 CPU 开销（若只占用全局 CPU 的 25%）、当前堆内存分配率、以及 GC 并发扫描的实际吞吐。当分配率上升或扫描较慢，Pacer 会动态调低触发比例，使得 GC 提前启动；若分配率较低，则延迟触发以节约 CPU 资源。

[M4.1] Channel 底层 hchan 环形缓冲区控制

Go 的管道在底层对应 C++ 风格的 hchan 结构体；\textbf{f1}.\textbf{flock} (锁): 保护 channel 内部所有字段的 mutex。对 channel 的任何 send 或 recv 操作都必须首先获取该锁。当 \textbf{f2}.\textbf{fbuf} (缓冲区): 环形队列数组，存储已发送但未被接收的对象。当 \textbf{f3}.\textbf{fsendx} / \textbf{recvx}: 记录当前环形数组的发送与接收游标索引。当 \textbf{f4}.\textbf{fbsendq} / \textbf{recvq}: 双向等待队列，分别挂载因为缓冲区满而阻塞的发送协程 \textbf{sudog} 链表，和因为缓冲区空而阻塞的接收协程 \textbf{sudog} 链表。

[M4.2] Channel 锁与 sudog 队列管理及直接拷贝优化

当协程读写阻塞时的底层控制流程如下：\textbf{f1}.\textbf{f}发送阻塞: G1 发送数据到已满 Channel。G1 创建一个 \textbf{sudog} 结构，将发送数据的内存物理地址绑定到该 \textbf{sudog}。然后将该 \textbf{sudog} 挂载在 \textbf{fchan}.\textbf{sendq} 队尾。调用 \textbf{gopark}() 将 G1 挂起。当 \textbf{f2}.\textbf{f}接收自愈直接拷贝: 当 G2 读取该空 Channel 时，发现 \textbf{sendq} 里有 G1 在等待。G2 获取锁，直接将 G1 \textbf{sudog}.\textbf{elem} 指向的内存数据物理拷贝（\textbf{memmove}）到 G2 本地的接收变量物理内存中，绕过了环形缓冲区 \textbf{buf} 的写入和读取，然后调用 \textbf{goready}() 将 G1 唤醒。

[M4.3] Select 多路复用非阻塞随机洗牌 selectgo

select 语句的并发处理由运行时 \textbf{selectgo}() 函数托管；\textbf{f1}.\textbf{f}随机洗牌: 进入 \textbf{selectgo} 后，系统使用伪随机算法对所有 case 的 channel 顺序进行洗牌重排（lock order 和 poll order），以确保每个 case 都有等概率被轮询，避免通道偏置。当 \textbf{f2}.\textbf{f}加锁与轮询: 按照 lock order 依次获取所有相关 channel 的锁。随后按照 poll order 遍历所有 case。若有 channel 就绪（可读/可写/有 default），则立即执行读写并释放所有锁。当 \textbf{f3}.\textbf{f}挂起挂载: 若所有 case 都不就绪，则为当前协程创建多个 \textbf{sudog}，分别挂载在所有 case channel 的等待队列中，随后解锁并挂起。当中任意一个 channel 就绪，它会唤醒 G，G 重新获取锁并清理其他未就绪的 \textbf{sudog} 节点。

[M4.4] Mutex 正常/自旋与饥饿模式判定模型

Go 的 \textbf{sync.Mutex} 采用精巧的双状态自适应升级控制模型以最大化 CPU 吞吐；\textbf{f1}.\textbf{f}正常模式 (Normal): 锁的等待者按 FIFO 顺序排队。但新到来的协程若正在 CPU 自旋（Spinning），会与队列头部的等待者竞争，由于自旋协程已在 CPU 上运行，其大概率抢锁成功。这有利于吞吐，但可能导致队列尾部协程无限饥饿。当 \textbf{f2}.\textbf{f}饥饿模式 (Starvation): 当队列头部的协程等待时间超过 1ms 时，Mutex 强制切换到饥饿模式。此时，锁的所有权直接从释放锁的协程传递给队列头部的等待者，新来的协程不会自旋且直接排在队尾。当 \textbf{f3}.\textbf{f}状态退役: 当获得锁的协程发现自己是队列中最后一个，或者其等待时间少于 1ms，Mutex 重新退回正常模式。

[M4.5] WaitGroup 与 Once 原子同步实现

\textbf{f1}.\textbf{f}WaitGroup: 包含一个计数器 \textbf{count}（记录当前未完成任务）和一个信号量池 \textbf{sema}。当调用 \textbf{Add}() 时通过原子操作修改计数器；调用 \textbf{Wait}() 时，若计数器不为 0，协程会将自己打包挂载在信号量 \textbf{sema} 的等待队列中并挂起。当计数器通过 \textbf{Done}() 归零时，主协程唤醒 \textbf{sema} 里的所有阻塞协程。当 \textbf{f2}.\textbf{f}Once: 包含一个 \textbf{done} 原子标志位和一个 \textbf{Mutex}。当调用 \textbf{Do}(\textbf{f}) 时，首先进行一次无锁的原子 Load 校验。若 \textbf{done} 为 0，则加锁，在锁内执行二次检查（Double Check），确保安全后调用函数 \textbf{f}()，最后使用原子操作将 \textbf{done} 设为 1 并释放锁。

[M5.1] Map 底层 hmap/bmap 内存排布

Go 的 Map 底层采用数组加链表形式的 Hash 表；\textbf{f1}.\textbf{f}核心结构体，包含桶数组指针 \textbf{buckets}、元素个数、当前扩容标记、以及指向溢出桶的指针 \textbf{extra}。当 \textbf{f2}.\textbf{f}hmap (bucket): 每个桶固定存储 8 个 key-value 对。结构上，它首先连续存储 8 个 \textbf{tophash}（哈希高 8 位，用于快速过滤），接着连续存储 8 个 key，最后连续存储 8 个 value。这种 key 和 value 分开存储避免了因 KV 大小不同导致的内存对齐空隙（Padding）。当 \textbf{f3}.\textbf{f}溢出桶 (Overflow Bucket): 当某个桶中的冲突 Keys 超过 8 个时，Go 分配一个新 \textbf{bmap} 并以指针挂载在旧桶的尾部。

[M5.2] Map 渐进式翻倍与等量扩容槽位迁移

Map 在两种情况下会触发扩容以防查表退化：\textbf{f1}.\textbf{f}翻倍扩容 (Double Size): 负载因子（元素个数/桶个数）超过 6.5，说明桶已拥挤。Go 申请双倍大小的新桶数组。当 \textbf{f2}.\textbf{f}等量扩容 (Same Size): 负载因子不高但溢出桶过多（很多空 Slot 且链表长，Go 申请等新桶，重新整理内存碎片）。当 \textbf{f3}.\textbf{f}渐进式迁移 (Evacuate): Go 不会在扩容时一次性迁移所有数据（导致卡顿），它在后续每次对 Map 执行 \textbf{delete} 或 \textbf{store} 时，顺带迁移当前操作的桶，以及顺序迁移一个 \textbf{oldbuckets} 游标指向的旧桶。旧桶里的 KV 被重新计算哈希并分流到新桶中。

[M5.3] 空接口 eface 与非空接口 iface 区别 itab

Go 接口是动态类型容器，在物理上分为两类：\textbf{f1}.\textbf{f}空接口 (空接口 \textbf{interface}): 用于承载任意类型。底层由两个指针构成：\textbf{type} 指向动态类型的反射元数据，\textbf{data} 指向具体的物理数据。当 \textbf{f2}.\textbf{f}非空接口 (非空接口): 底层由 \textbf{itab} 指针和 \textbf{data} 指针构成。当 \textbf{f3}.\textbf{f}物理结构: 保存了接口静态类型定义、动态具体类型的 \textbf{type} 元数据，以及核心的虚函数表指针列表 \textbf{fun}。在类型断言或接口转换时，运行时会在全局哈希表中查找并复用对应的 \textbf{itab} 结构，降低动态生成开销。

[M5.4] 接口动态分发虚函数调用开销与去虚拟化

当通过接口调用方法（如 \textbf{reader.Read}(\textbf{buf})）时：\textbf{f1}.\textbf{f}动态分发: 运行时必须先读取 \textbf{iface}.\textbf{itab}，偏移检索 \textbf{fun} 数组中的函数指针，然后将 \textbf{iface}.\textbf{data} 放入第一寄存器（作为 receiver）跳转执行。这产生了一次间接内存寻址与无法被 CPU 分支预测内联的开销。当 \textbf{f2}.\textbf{f}去虚拟化 (Devirtualization): Go 编译器在高级优化 Passes 中，如果能够通过逃逸分析和控制流静态推导出当前接口的具体实现类型，它会将动态分发调用改写为直接对具体类型的静态直接调用，从而允许内联（Inlining），大幅提升执行速度。

[M6.1] Defer 三种编译路径堆/栈/开放编码

为了降低 defer 带来的历史性能开销，Go 演进并实现了三种共存的编译策略：\textbf{f1}.\textbf{f}堆分配 (Heap-allocated): Go 1.13 之前，在遇到 defer 时，调用 \textbf{deferproc}() 在堆上分配一个 defer 实例挂载在 G 的链表上。函数退出时调用 \textbf{deferreturn}() 执行，开销极大。当 \textbf{f2}.\textbf{f}栈分配 (Stack-allocated): Go 1.13 引入。如果 defer 未在循环内使用，直接在当前函数栈帧上分配 defer，性能提升 10 倍。当 \textbf{f3}.\textbf{f}开放编码 (Open-coded): Go 1.14 引入。在编译期如果 defer 数量固定且在非循环中，编译器直接将 defer 的执行逻辑内联到函数的所有 return 出口处，并通过一个本地 \textbf{bimask} 标记来控制执行顺序。这消除了全部运行时 defer 结构体的创建开销，达到了零运行时损耗（Zero-cost）。

[M6.2] Panic/Recover 嵌套链式处理 panic 状态转移

\textbf{f1}.\textbf{f}panic 结构: 当触发 panic 时，系统在当前 G 的头部挂载一个 \textbf{panic} 结构体，内部持有 \textbf{recovered}（是否被恢复）和 \textbf{link}（指向父 \textbf{panic} 的指针，支持嵌套）。当 \textbf{f2}.\textbf{f}执行 Defer 链: Go 挂起当前正常执行流，开始以 LIFO 顺序遍历并执行当前 G 的 \textbf{defer} 链表。当 \textbf{f3}.\textbf{f}Recover 阻断: 如果在执行某个 defer 时调用了 \textbf{recover}()，系统会将 \textbf{panic}.\textbf{recovered} 设为 \textbf{truec}，并读取现场 \textbf{g}.\textbf{sched} 恢复寄存器上下文，直接跳转到当前 defer 调用函数之后的返回流，阻断 panic 向上抛出并优雅自愈。

[M6.3] 协作式系统调用包装 Syscall 调度分离

\textbf{f1}.\textbf{f}包装函数: 当 G 发起阻塞系统调用（如 \textbf{syscall.Read}）时，包装函数会触发 \textbf{entersyscall}()。Go 将当前 P 的状态设为 \textbf{Pgcstop}，并将 P 与当前执行 M 进行物理绑定分离。M 随后陷入内核态系统调用。当 \textbf{f2}.\textbf{f}sysmon 接管: 后台 \textbf{sysmon} 扫描发现 P 处于系统调用状态且对应的 M 阻塞，它会强行夺取 P 的所有权，并分配给其他空闲的 M（或者新建 M）去执行 P 队列里的其他 G，实现高吞吐。当 \textbf{f3}.\textbf{f}退出系统调用: 当系统调用退出时，M 触发 \textbf{exitsyscall}()。它首先尝试原子抢占原先的 P 或其他空闲的 P。若抢占成功则继续执行 G；若失败，将 G 标记为 \textbf{runnable} 并扔进全局 G 队列，M 则解绑并休眠。

[M6.4] Cgo 调用双向栈切换与运行时开销

由于 C 语言没有协程概念且其执行不被 Go 运行时管理，Cgo 跨越了两个截然不同的运行时限界：\textbf{f1}.\textbf{f}栈切换: Go 协程栈是动态增长且极小的（2KB）。而 C 代码需要大且固定的系统线程栈（8MB），因此，在通过 Cgo 调用 C 函数前，运行时必须调用 \textbf{asmgocall}()，将当前寄存器 SP 切换到系统物理线程的 M_0 栈上。当 \textbf{f2}.\textbf{f}调度冻结: 为了防止 Go 的 GC 线程在 C 执行时移动内存，Cgo 会临时将 G 标记为 \textbf{syscall} 状态，使 GC 认为其已挂起；同时将 C 指针指向的数据固定在原处。当 \textbf{f3}.\textbf{f}栈切换: 频繁的栈切换、寄存器保护与恢复，以及跨语言屏障，导致一次 Cgo 调用通常需要耗费数十纳秒，比纯 Go 函数调用慢数十倍。

Zone T1: Go debug flags & tracing

\textbf{GODEBUG=schedtrace=1000}：开启调度器追踪。每隔 1000 毫秒在终端打印一行调度器摘要信息，包含全局队列大小、各 P 的本地队列大小以及活跃的 M/P 数量。当 \textbf{GODEBUG=gctrace=1}：开启垃圾回收监控追踪。每次 GC 结束后打印一行详细数据，包含回收比例、存活堆大小、STW 停顿时间、写屏障开启开销及 GC 实际占用的 CPU 占比。当 \textbf{go build -gcflags="-m -l"}：控制编译器诊断。-m 开启编译器逃逸分析静态推导诊断，打印出哪些对象逃逸到了堆上，哪些停留在栈上；-l 禁用内联优化，便于精准追踪函数堆栈物理调用。

Zone T2: Go assembler & write barrier listings

在 Go 底层汇编（Plan 9 Assembly 格式）中，可以清晰观测到运行时底层原语的插入：\textbf{f1}.\textbf{f}栈空间检查 (Stack Split check)：在每个 Go 函数入口，都会出现以下特征汇编以判定协程栈是否溢出：\textbf{MOVQ}(\textbf{TLS}), \textbf{CX} // 获取当前 Goroutine 指针 \textbf{g} \textbf{CMPQ} \textbf{SP}, 16(\textbf{CX}) // 比较当前栈指针 \textbf{SP} 与 \textbf{g}.\textbf{stackguard0}（偏移量 16）\textbf{JLS} 24 // 若 \textbf{SP} <= \textbf{stackguard0}，跳转到 \textbf{newstack} 调用执行栈拷贝 \textbf{f2}.\textbf{f}混合写屏障插入模式 (Write Barrier active check)：当代码向一个指针对象写入值时（例如 \textbf{p}.\textbf{next} = \textbf{node}），C2 级别的 Plan 9 汇编不会直接执行 \textbf{MOVQ}，而是会先判定写屏障状态：\textbf{LEAQ} \textbf{runtime.writeBarrier}(\textbf{SB}), \textbf{R11} \textbf{CMPB}(\textbf{R11}), \textbf{0} // 检查写屏障 \textbf{active} 状态位是否为 1 \textbf{JEQ} \textbf{L_direct_write} // 若为 0，直接跳转到 \textbf{L_direct_write} 执行普通 \textbf{MOVQ} \textbf{CALL} \textbf{runtime.gcWriteBarrier}(\textbf{SB}) // 若为 1，强行调用写屏障方法，将被覆盖的旧值与新值置灰